Companion Bot
By
FiveOhms

Team Members:
Khoo Lay Yang
Ang Jin Jin
Joey Wong Zho Yew
Ng Anson
Chung Chee Kin
…

Executive Summary

This project addresses the limitation of conventional stationary voice assistants, which lack mobility and environmental awareness. While devices such as Alexa or Google Home provide voice interaction, they cannot physically reposition themselves or visually perceive their surroundings. As smart environments continue to evolve, there is an increasing demand for intelligent systems that combine mobility, voice interaction and visual sensing. The proposed system functions as a mobile smart-home assistant capable of following users, capturing visual information, and accessing online resources to provide enhanced interactive support.

To address this limitation, a mobile smart assistant robot was developed using Raspberry Pi 5 as the main central processing unit, integrated with YOLO-powered real-time human detection and tracking. The system combines voice recognition, camera-based image capture, and motorized movement into a single platform. Upon receiving voice input, the robot can either generate a voice response or capture images of its surroundings, depending on the sequential decision logic that determines the final action.

This system was implemented using a modular Python-based software architecture on Raspberry Pi OS. Hardware components include a camera module, microphone, speaker, alongside with 4 motor driver that will be used and tested iteratively to ensure smooth and stable operation.

Testing demonstrated reliable voice interaction, successful image capture functionality, and stable mobility performance in indoor environments. The project validates the feasibility of integrating speech processing, computer vision, and autonomous movement within a compact edge-computing platform, with potential applications in assistive robotics, surveillance systems, and smart home automation.

Design & Methodology

Section 1. Project Concept Development

The initial concept was developed based on the limitations of conventional stationary voice assistants, which are unable to move, perceive its surrounding, or adapt to user position. The objective of this design was to create a mobile smart assistant with environmental awareness and the ability to access the internet for up-to-date information upon user request, allowing it to operate more naturally within indoor environments.

To achieve this, the project focused on integrating multimodal interaction, including voice recognition, visual sensing, and autonomous movement. Additionally, this system was designed as an edge-computing platform so that processing could be performed locally on the robot rather than relying solely on cloud services. Based on these concepts, the Companion Bot was designed to follow user around with the help of Computer Vision while responding to voice commands and capturing visual information from its surroundings.

Section 2. Overall System Architecture

Our system architecture will be split into 5 main subsystems:

i. Processing Subsystem

At the core of the design, Raspberry Pi 5 is used as the central controller. It coordinates data from the camera and microphone, executing AI-based human detection, and generates movement commands for the motor. All processing is centralized within a single system to enable real-time perception, decision making, and movement in indoor environments.

ii. Vision Subsystem

This subsystem functions as the visual sensing components of the Companion Bot. It enables a continuous video frame acquisition using a camera module connected to Raspberry Pi. A pretrained YOLO (You Only Look Once) object detection model is used to identify and track human in real time.

The YOLO model outputs bounding boxes for detected objects within the camera frame. The corresponding coordinates are extracted and used as the basis for tracking. From these bounding box coordinates, the centre point of detected person is calculated and compared with the centre of the camera frame to determine the direction of movement for the robot.

iii. **Audio Subsystem**

This subsystem uses a microphone module to capture user speech input. The audio signal is processed and converted into digital text using a speech recognition library. The transcribed text is then passed to a Large Language Model (LLM) via the Ollama framework for semantic interpretation.

Instead of relying on predefined command phrases, the LLM analyses the contextual meaning of the input and determines the appropriate system action, such as generating a response, capturing an image, or performing an internet-based query. The subsystem also supports audio output through a speaker module to provide responses to the user.

iv. Mobility Subsystem

The mobility subsystem provides physical movement to the Companion Bot. An L298N motor driver is used to control for DC motors, as it supplies sufficient current and electrically isolates the Raspberry Pi from motor load fluctuation. The motors are controlled via GPIO pins, allowing the software to regulate speed and direction using digital signals and Pulse Width Modulation (PWM) control.

Directional movement is determined based on the tracking output from the vision subsystem, enabling the robot to adjust its motion and follow the detected user. PID (Proportional Integral Derivative) control is applied to refine motor control and ensure smooth, precise movement. The PID controller adjusts motor speed and direction dynamically based on feedback from the tracking system. This allows the Smart Bot to correct its path with minimal overshooting or oscillation, achieving more stable and accurate following behaviour.

v. Network Subsystem

The network subsystem utilizes the built-in Wi-Fi module of the Raspberry Pi 5 for internet connectivity. This enables the Smart Bot to process internet-based queries and support smart assistant functionalities that require online access.

Section 3. Human Tracking Methodology (YOLO-Based Detection)

The human tracking functionality of the Companion Bot is implemented using a camera-based computer vision pipeline combined with an object detection model, The objective of this subsystem is to detect a person in the camera frame in real time and generate control signals that allow the robot to follow the detected user.

A pretrained YOLO object detection model is used to perform real-time human detection on human detection on each frame that were captured by the Pi Camera. The model analyses each frame and outputs bounding boxes around detected human body regions. For tracking, the detections associated with human body parts are selected, and the bounding box coordinates of the most relevant region are extracted. The centre of this bounding box is computed and treated as the approximate position of the user within the image.

This estimated position is compared with the centre of the camera frame, and the horizontal offset is used to generate commands for the robot, allowing it to turn and move in respect to user's direction.

Section 4: Software Architecture

The software architecture of the Companion Bot is designed using a modular structure, where separate components are responsible for voice processing, vision-based human tracking, motion control, and auxiliary functions such as image capturing and network access. All functionalities are implemented in Python and executed directly on the Raspberry Pi, enabling real-time coordination between sensor inputs, YOLO-based human detection, and motor control output. Each module performs a specific task independently while maintaining structured communication through well-defined data flows. This modular integration forms a complete control pipeline, transforming sensor input into coordinated robotic action. Due to computational constraints of edge-based processing, the system adopts a sequential execution model rather than a fully parallel architecture.

4.1 Real-Time Operational Workflow

The Smart Bot operates using a continuous real-time processing loop that integrates vision-based human detection with motor control. Upon system initialization, the Raspberry Pi loads the YOLO object detection model, initializes the camera stream, and configures the GPIO pins connected to the motor driver. Once initialization is complete, the system continuously captures video frames and processes them sequentially.

Each captured frame is passed through the detection model to identify objects within the scene. The detection results are filtered to isolate the "person" class, which serves as the tracking target. Based on the detection output, movement commands are generated and transmitted to the motor driver to adjust the robot's direction and position accordingly.

This tracking mechanism operates continuously and independently of the voice subsystem. The mobility of the Smart Bot is fully controlled by the vision detection output, ensuring autonomous following behavior without requiring activation commands.

4.2 Vision-Based Human Tracking Algorithm

The human tracking mechanism is implemented using a pretrained YOLO object detection model. For each video frame, the logic will be as follow:

i. The frame is captured from the Pi Camera module.
ii. YOLO performs inference and generates bounding boxes for detected objects.
iii. Detections classified under the "person" category are selected
iv. The bounding box coordinates (x, y, width, height) are extracted
v. The centroid of the detected person is calculated using:

$$center\ point = x + \frac{width}{2} \tag{1}$$

The computed centroid is then compared with the horizontal centre of the camera frame. A directional offset is calculated to determine motor adjustment.

Movement logic is defined as follows:

- If the detected person move towards the left of the frame centre beyond a predefined threshold, the robot rotates left.
- If positioned right of the centre, the robot rotates right.
- If aligned near the centre region, the robot moves forward
- If no person is detected, the robot will……

To prevent unstable movement due to fluctuating detections, minor centroid variations within a tolerance range are ignored.

4.3 Motor Control Strategy

Motor control is implemented using the L298N motor driver with Pins connected to the Raspberry Pi's GPIO Pins. Digital output signals determine direction, while Pulse Width Modulation (PWM) is used to regulate motor speed.

The directional correction signals generated by the tracking algorithm are translated into corresponding motor commands:

- Left motor slowed + Right motor accelerated for right turns
- Right motor slowed + Left motor accelerated for left turns
- Both motors same speed for forward movement

This creates a feedback-based control mechanism, where visual input continuously influences motor output. Although a PID controller is not implemented, the system operates as a basic closed-loop tracking system based on real-time visual feedback.

4.4 Voice and LLM-based Decision Processing

The Companion Bot utilizes a speech-to-text module to convert spoken user input into textual form. The transcribed text is then passed to a locally hosted Large Language Model (LLM) via the Ollama Framework for semantic interpretation.

Instead of relying on predefined command phrases, the LLM processes natural language input and determines the appropriate system action based on contextual understanding. Depending on the interpreted intent, the system will decide the following actions:

a. Perform an internet based query
b. Trigger image capture
c. Generate a conversational response
d. Execute auxiliary system functions

This architecture enables flexible natural language interaction rather than rigid command-based control.

Due to computational constraints, LLM inference is executed sequentially and does not run concurrently with the continuous vision tracking process.

4.5 System Performance Considerations

To maintain stable real-time operation, several optimization strategies were implemented:

- Input frame resolution was adjusted to balance detection accuracy and processing speed.
- Detection confidence thresholds were applied to reduce false positives.
- Minor centroid fluctuations were ignored to precent motor jitter.
- Sequential processing logic was deployed to avoid resource conflict between detection and voice processing.

These safety measures ensure a smooth tracking behaviour and reliable system performance during indoor operation.

5. Development Stages

The development of the Companion Bot was carried out through a structured and systematic process to ensure proper integration between hardware and software components.

The first stage involves a requirement analysis, where the functional objectives of the system were defined. These included real-time human detection, autonomous following behaviour, voice command processing, image capture functionality, and internet connectivity. System constraints such as processing capability and power management were also considered during this phase.

Afterwards, the second stage consisted of hardware assembling and wiring. The Raspberry Pi 5, camera module, L298N motor driver, DC motors, microphone, and speaker were individually tested for any manufacturing fault before integrating into one whole system. After testing individually and integrating all components into one complete system, final testing for proper electrical connections and signal flow were conducted.

Following hardware setup, software development was conducted using Python on Raspberry Pi OS. Separate modules were implemented for vision detection, tracking logic, motor control, and voice processing. The YOLO object detection model was deployed for real-time human detection.

Integration testing was then performed to ensure that all subsystems functioned cohesively. Communication between modules was validated, and real-time tracking performance was evaluated. The testing was conducted under indoor conditions to observe detection accuracy, tracking stability, and system responsiveness. Finally, system optimization was performed to improve processing efficiency and reduce instability during operation.

6. Design Justification

Key hardware and software components were selected based on performance requirements compatibility, and suitability for edge-based artificial intelligence processing.

Raspberry Pi 5 was selected as the central controller due to its high processing capability, camera interface support, and ability to execute AI models locally. Its Linux-based operating system provides flexibility for software development, while Python compatibility simplifies implementation of computer vision and speech processing libraries.

The YOLO object detection model was chosen for human detection due to its real-time inference capability and efficient single-pass detection architecture. YOLO enables object localization and classification within a single forward pass, making it suitable for edge devices with limited computational resources.

Edge processing was adopted to reduce dependency on cloud services. This approach minimized latency, improves privacy by keeping data local, and enhances system responsiveness for real-time tracking applications.

Overall, the selected architecture balances computational efficiency, detection accuracy and practical implementation feasibility.

7. Challenges and Solutions

During development, several technical challenges were encountered and addressed through targeted optimization strategies.

High CPU usage was observed during YOLO inference, particularly during continuous real-time detection. This issue was mitigated by reducing the input frame resolution, which improved processing speed while maintaining acceptable detection accuracy.

Detection instability occasionally caused fluctuating bounding box outputs, resulting in unstable movement behaviour. A confidence threshold filter was implemented to discard low-probability detections and improve tracking reliability.

Motor jitter was observed when rapid directional corrections were triggered due to minor centroid variations. To address this, a tolerance threshold was introduced to ignore small positional deviations, thereby smoothing motor control behaviour.

Furthermore, power instability occurred during testing with a single power bank. The issue was likely caused by insufficient current supply to meet system demand. This problem was resolved by utilizing two separate power banks to distribute the load more effectively.

These optimizations enhanced the reliability and real-time performance of the Smart Bot during continuous indoor operation.

Results & Testing Evidence

1. Testing Methodology

System testing was conducted in a controlled indoor environment to evaluate detection accuracy, tracking stability, voice command recognition, and overall system responsiveness. The Companion Bot was placed in a room with typically lighting conditions and a single human subject positioned at varying distances and angles relative to the camera.

Testing was divided into four primary evaluation categories:

i. Human detection accuracy
ii. Movement Stability and Tracking Reliability
iii. Voice Command functionality with accuracy of response
iv. Overall system performance and stability

Each subsystem was first validated independently before conducting integrated system testing.

2. Human Detection Performance

The YOLO-based detection model successfully identified the “person” class in real-time video frames under normal indoor lighting environments. Detection was consistent when the subject was positioned within the camera’s field of view and at moderate distances.

Detection reliability decreased slightly under low-light conditions and when the subject was partially blocked. However, the confidence threshold filtering mechanism reduced false detections and maintained stable tracking behaviour during normal operation.

The system demonstrated stable detection across repeated test cycles without model failure or crash.

3. Tracking and Mobility Performance

During testing, the robot successfully followed the detected human subject using centroid-based directional correction. The motor control logic allowed the Companion Bot to perform following actions:

- Rotate left or right when subject moved laterally
- Move forward and stop at a predefined distance before colliding onto the subject or object in front of it.
- Stop when no person was detected nearby.

4. Voice and Auxiliary Function Testing

Voice interaction testing was conducted to evaluate the system's ability to process natural spoken input and determine appropriate actions using a Large Language Model (LLM) executed through the Ollama framework. During testing, users provided conversational and task-oriented inputs under typical indoor noise conditions.

Audio signals were captured through the microphone and converted into text using a speech-to-text module. The transcribed input was then forwarded to the local LLM for semantic interpretation. Instead of relying on predefined command phrases, the LLM analysed the contextual meaning of the input and determined whether the request required one of the following:

1. Internet-based information retrieval
2. Image capture functionality
3. Conversational response generation
4. Other auxiliary system actions

Testing demonstrated that the system was capable of interpreting open-ended spoken input and dynamically selecting appropriate actions. Queries requiring web access were successfully processed through the network subsystem, while local tasks such as image capturing were executed without any visible instability.

Due to the sequential execution model, minor response delays were observed during LLM inference. However, system functionality remained stable, and no crashes occurred during extended testing sessions.

Overall, the integration of LLM-based decision processing enabled flexible natural language interaction, significantly enhancing the Companion Bot's usability compared to fixed command-based systems.

5. Expected vs Actual Results

At the beginning of development, the expected system behaviour included continuous real-time human detection, stable autonomous following, accurate interpretation of natural language input, and reliable execution of auxiliary functions such as image capture and internet queries.

During actual testing, the system achieved consistent human detection under normal indoor lighting conditions. The Companion Bot successfully followed a detected user by adjusting its direction based on bounding box centroid positioning. Minor oscillations were observed when the subject moved rapidly across the camera frame. However, threshold smoothing reduced unstable motor responses.

For voice interaction, the LLM-based processing successfully interpreted natural language input without any predefined command structures. The system demonstrated the ability to differentiate between conversational queries and action-based requests. Internet-dependent tasks was executed correctly through the network subsystem.

Although the system performed according to its core objectives, slight delays were observed during LLM inference due to sequential processing constraints. Despite this, system stability was maintained significant unexpected error throughout the whole testing.

Overall, the actual system performance closely aligned with the initial expectations.

6. Limitations of Current System

Several limitations were identified during system evaluation.

First, due to sequential execution model, the system could not process vision tracking and LLM-based voice interpretation simultaneously. This introduced brief pauses when switching between computationally intensive tasks.

Next, tracking accuracy was limited by camera field of view. Subjects positioned too far from camera or outside the detection range could not be reliably tracked.

Finally, the overall system performance was constrained by the processing capacity of the Raspberry Pi 5. While capable of running both YOLO and an LLM module locally, resource limitations required optimization strategies such as reduced frame resolution and detection filtering.

These limitations represent practical constraints of edge-based embedded AI systems.

7. Summary of Performance

The Companion Bot successfully achieved its primary objectives of integrating vision-based human tracking, LLM-driven voice interaction, image capture functionality, and internet access within a single edge-computing platform.

Testing confirmed that the system could:

- Detect and track a human subject in real time
- Adjust motor movement autonomously based on visual feedback
- Interpret natural language input using a locally hosted LLM
- Execute both local and internet-based tasks reliably

Despite computational constraints and environmental limitations, the system demonstrated stable and repeatable operation under normal indoor conditions.

The results validate the feasibility of combining computer vision, large language model inference, and autonomous mobility within a compact Raspberry Pi-based robotic platform.

Bill of Materials (BoM)

No.	Component	Quantity	Price (RM)	Notes
1.	Wheel/4pcs	1	6	
2.	Ultrasonic Sensor HC-SR04	3	13.6	
3.	18650 Battery Holder	1	6.43	
4.	Omnidirectional Microphone	1	34.71	
5.	7 inch IPS Touch Screen	1	90.18	
6.	Pi Camera	1	79	
7.	Stereo USB Power and Signal Speaker	1	35	

Future Improvements

Although the Companion Bot successfully integrates vision-based tracking, LLM-driven interaction and autonomous mobility, several enhancements can be implemented to further improve system performance, efficiency, and functionality.

1. Computational Hardware Upgrade

During testing, CPU utilization on the Raspberry Pi 5 frequently approached 100% usage capacity during continuous YOLO inference and LLM processing. This indicates that the current platform is operating at its computational limits.

To enable advanced features such as true parallel processing, smart and mobile assistant, higher-resolution object detection, or even a larger LLM model for better response, an upgraded processing platform would be required. Potential hardware improvements include:

- Migration to a higher-performance single-board computer
- Use of GPU-enabled embedded systems such as NVIDIA Jetson platforms
- Integration of external AI accelerators (Coral TPU)
- Offloading LLM inference to a remote server

Upgrading the computational hardware would significantly improve inference speed, reduce latency, and allow simultaneous execution of vision and language processing tasks.

2. AI Model Optimization

Instead of increasing hardware capability, performance can also be improved through software optimization. Lightweight or quantized versions of the YOLO model and LLM could be deployed to reduce computational load at the cost of accuracy of response.

Techniques such as model pruning, reduced input resolution, and parameter-efficient inference would improve responsiveness while maintaining acceptable accuracy. This approach would enhance system efficiency without requiring major hardware changes.

3. Parallel Processing Architecture

The current system operates using a sequential execution model due to computational constraints. A future improvement would involve implementing parallel processing using multi-threading or asynchronous task handling. This would allow simultaneous execution of vision tracking and LLM-based voice processing, improving responsiveness and reducing task-switching delays.

4. Enhanced Tracking and Motion Control

The present tracking system uses centroid-based directional logic. Future improvements could include the implementation of a PID controller to achieve smoother and more stable movement. Additionally, distance estimation techniques based on bounding box scaling or depth sensors could enable more precise following behaviour.

5. Obstacle Detection and Navigation

The current system focuses primarily on human tracking. The integration of additional sensors such as LiDAR or depth cameras would improve environmental awareness and allow obstacle avoidance capabilities. This would enhance safety and enable operation in more complex indoor environments.

6. Advanced Conversational Capabilities

While the LLM enables natural language interpretation, future versions could incorporate contextual memory, multi-turn dialogue handling, and task persistence. These improvements would allow the Companion Bot to function more effectively as a long-term interactive assistant.

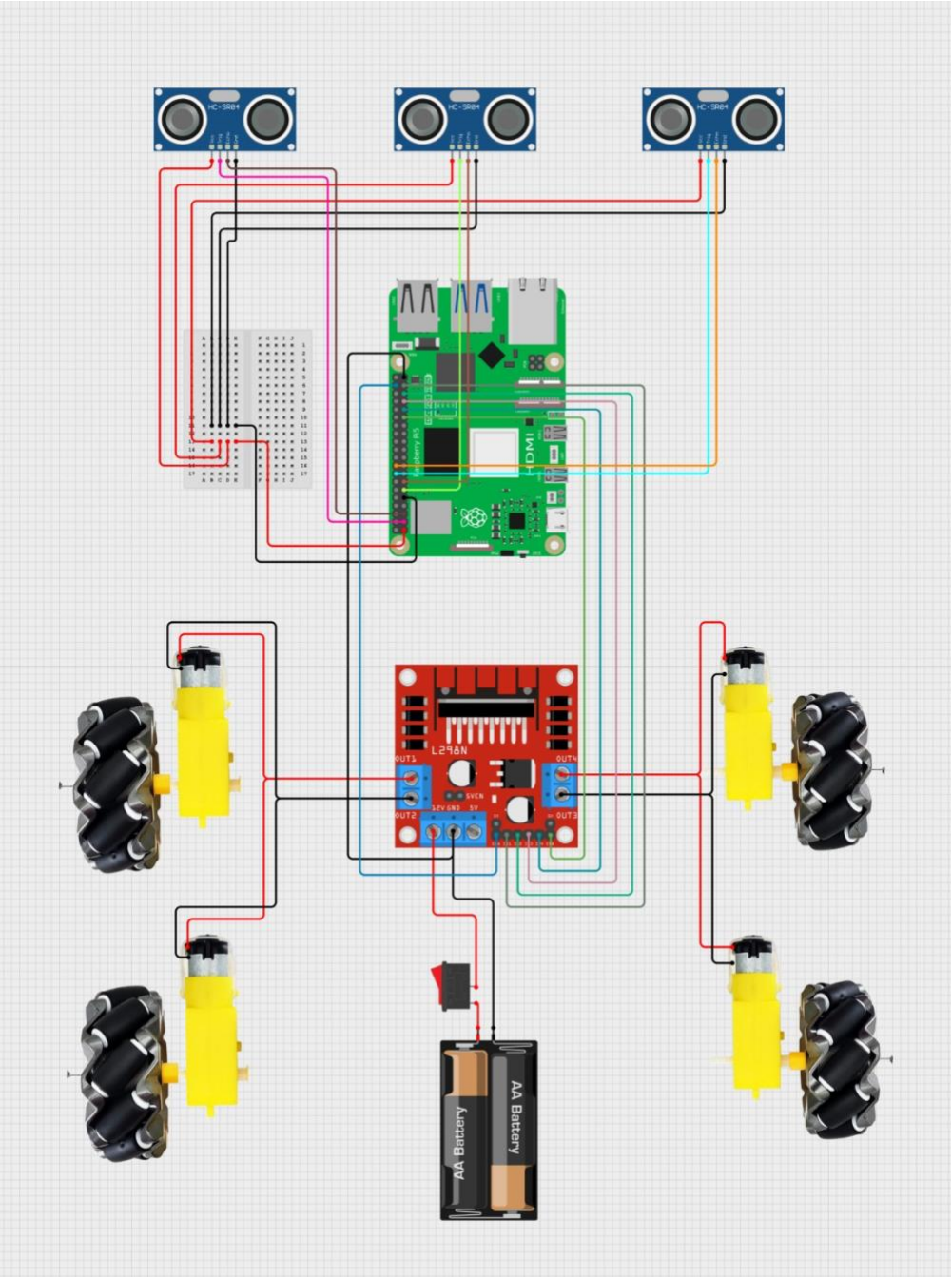
Such enhancements may require additional computational resources or cloud-based inference depending on model complexity.

9.7 Energy and Mechanical Optimization

Battery efficiency and thermal management can be further optimized to improve system durability during extended operation. Additionally, improvements in chassis design, motor mounting, and vibration damping would enhance physical stability and motion smoothness.

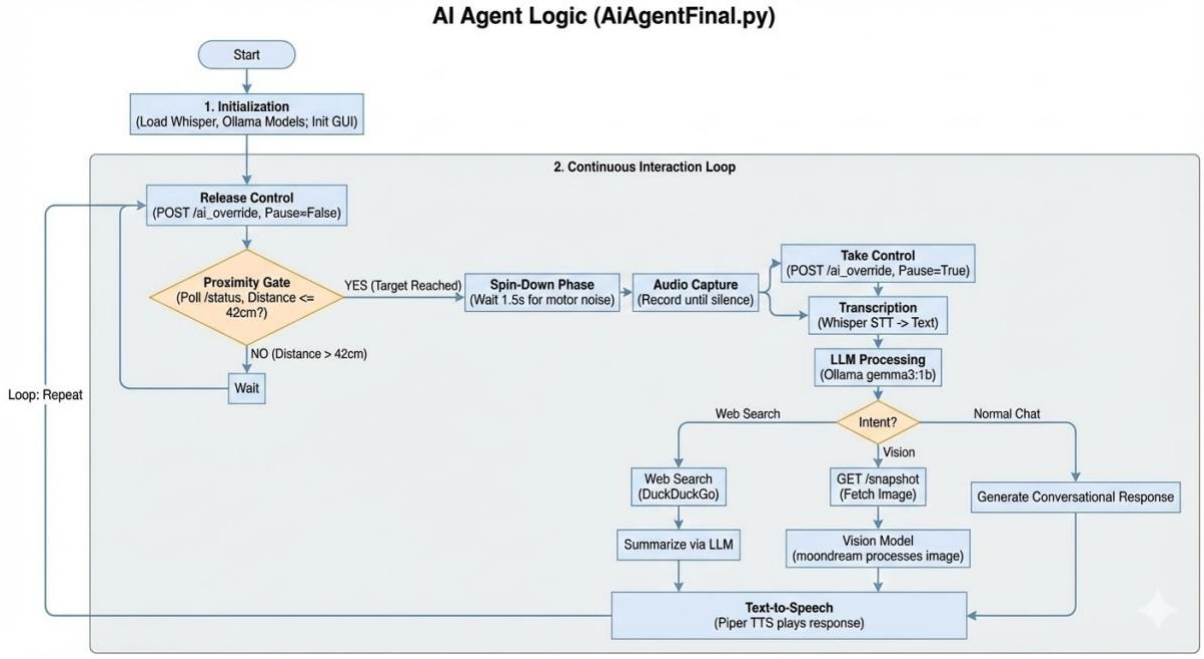
Appendices

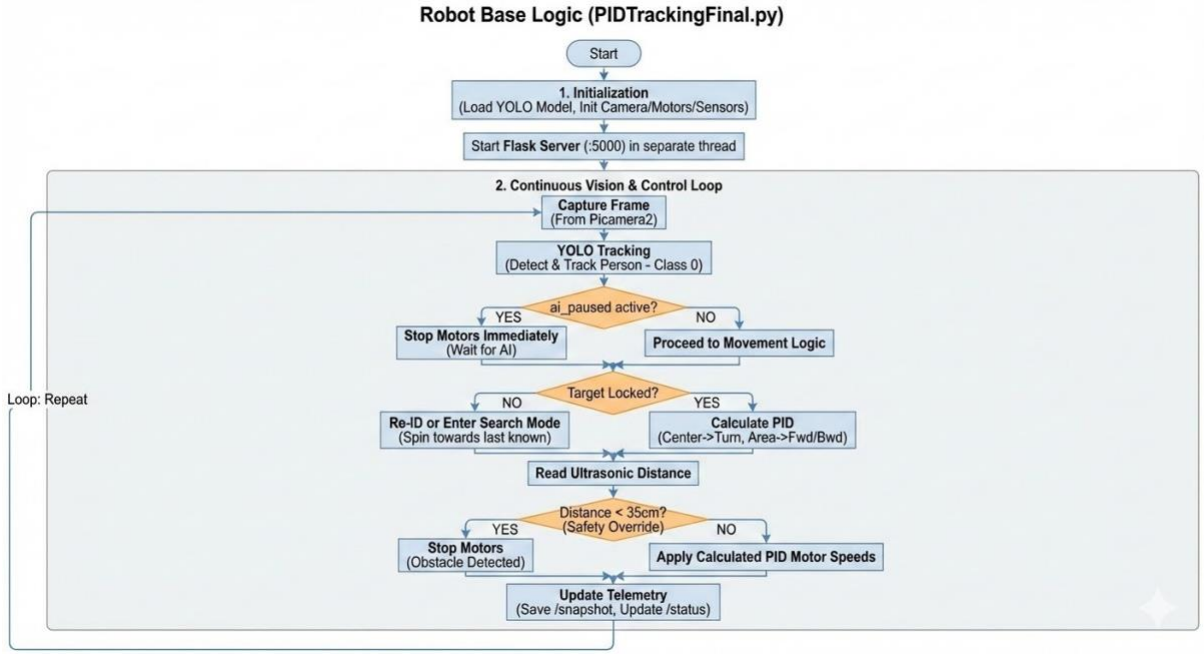
- Circuit schematics and wiring diagrams

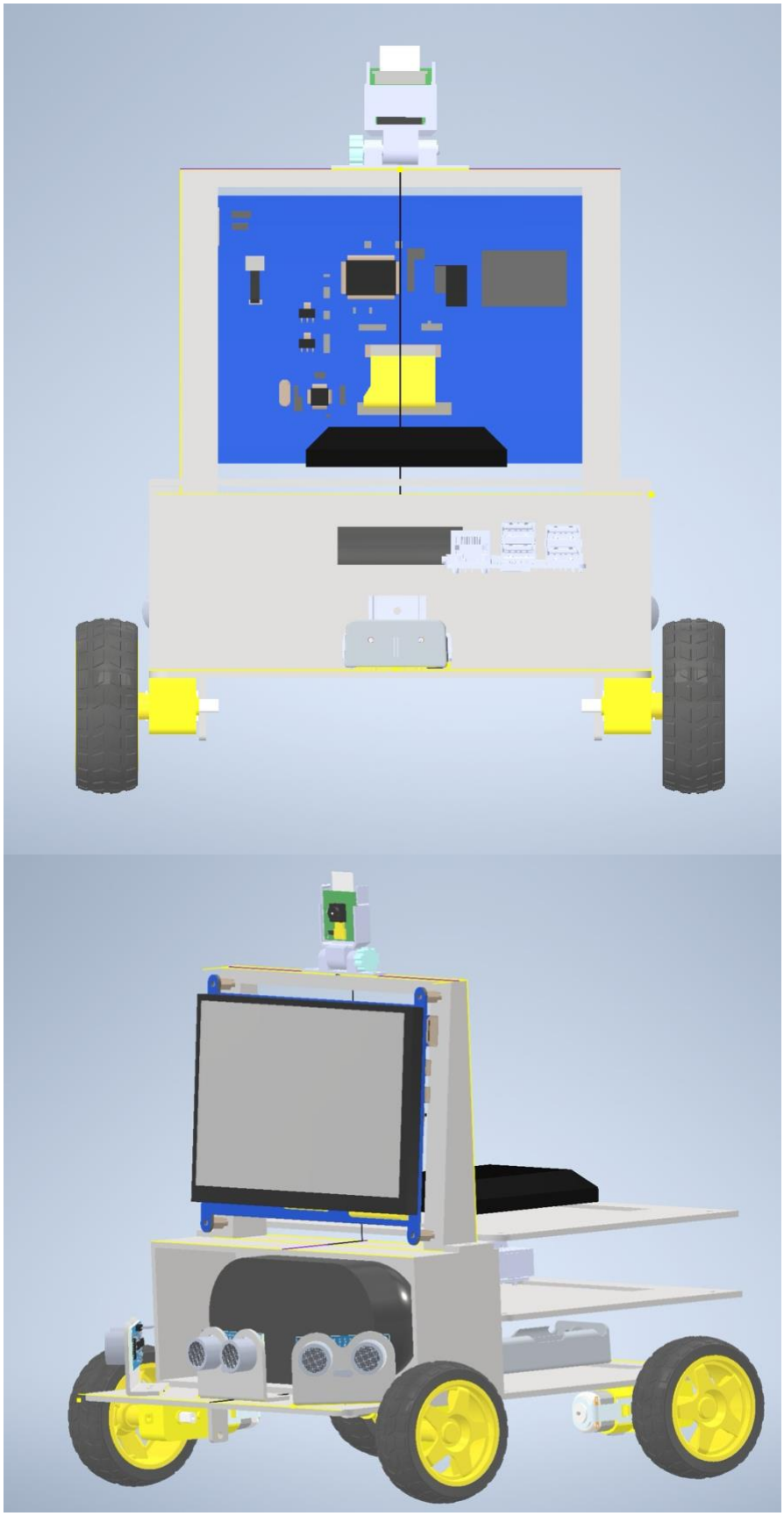


For Pin Configuration:
https://docs.google.com/document/d/1u00O6Ceb2PznJBq0L03jv8s-gKVp-_E0KYxfATfQu88/edit?usp=sharing

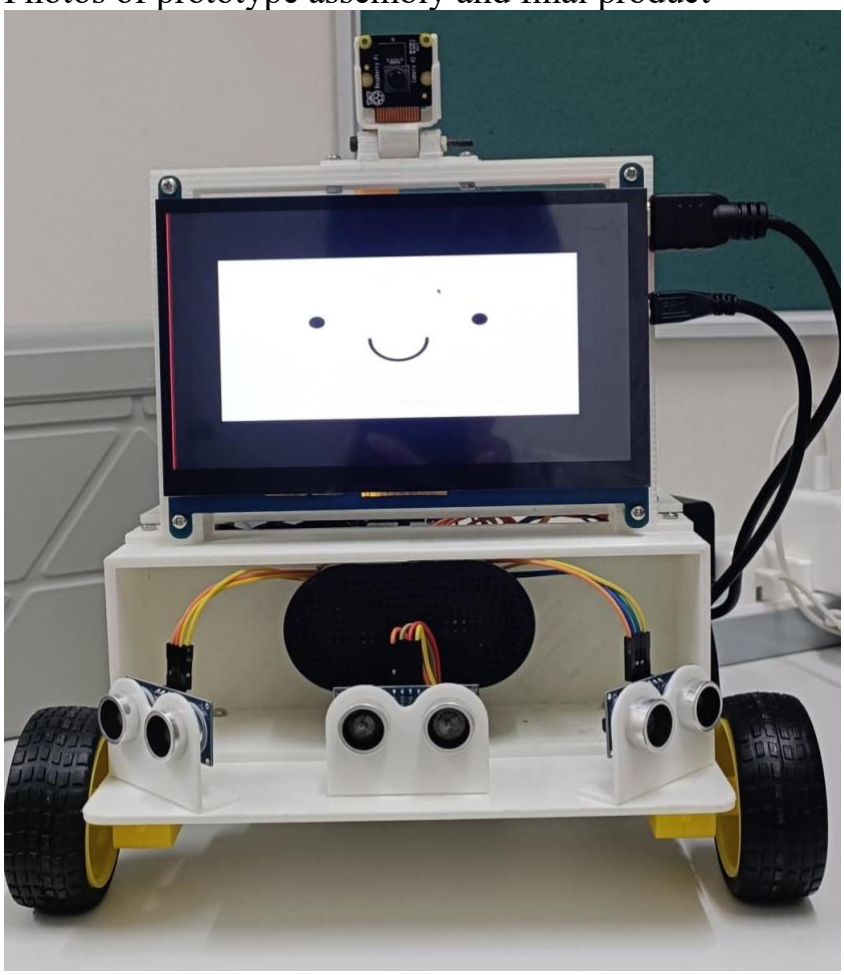
- Flowcharts, block diagrams, and 3D designs







- Photos of prototype assembly and final product



- Any additional references, datasheets, or documentation

Documentation:

- Raspberry Pi 5:
  - chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/https://pip-assets.raspberrypi.com/categories/892-raspberry-pi-5/documents/RP-008348-DS-5-raspberry-pi-5-product-brief.pdf?disposition=inline
  - chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/https://pip-assets.raspberrypi.com/categories/892-raspberry-pi-5/documents/RP-008347-DS-1-raspberry-pi-5-mechanical-drawing.pdf?disposition=inline
  - chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/https://pip-assets.raspberrypi.com/categories/892-raspberry-pi-5/documents/RP-008370-DS-1-rp1-peripherals.pdf?disposition=inline

- Raspberry Pi Camera Module 2 NoIR
  - https://www.raspberrypi.com/documentation/accessories/camera.html
- Ollama
  - https://docs.ollama.com/
- Ollama (gemma3)
  - https://ollama.com/library/gemma3
- Ollama (moondream:v2)
  - https://ollama.com/library/moondream:v2
- Whisper (stt)
  - https://github.com/openai/whisper
- Piper (tts)
  - https://github.com/OHF-Voice/piper1-gpl
- DuckDuckGo API
  - https://github.com/luminati-io/duckduckgo-api/blob/main/README.md
- Ultralytics YOLO11 (human detection)
  - https://docs.ultralytics.com/models/yolo11/